

Kinematics of the myCobot 280 — A Worked Book

Forward & inverse kinematics, the velocity-propagation Jacobian, manipulability, and the code — in reference notation

For: Abhishek Roy

2026

Preface — how to read this book

This book explains the mathematics **and** the code of the myCobot 280 projects, in the **notation of your reference notes** (modified Denavit–Hartenberg, the velocity-propagation method). Read a chapter, then open the file it names and match each equation to the lines of code.

Two kinds of callouts appear throughout:

Key point. The one idea you must not miss.

Meaning. A plain-language reading of the symbols right after each equation.

Notation used throughout (from your reference notes):

$${}^i_{i-1}T \quad {}^i\omega_i \quad \dot{\theta} \quad \hat{Z}$$

A **leading super/subscript** denotes the frame: the first symbol above is the transform **from frame $i-1$ to frame i** ; the second is an angular velocity **of link i , written in frame i** . A **dot** is a time derivative (joint speed), a **hat** is a unit vector (the z-axis).

Chapter 1 — The robot and its frames

The myCobot 280 is a 6-joint (6-DOF) arm; every joint is **revolute** and turns about its own z-axis. To do kinematics we attach a coordinate **frame** to each link and describe how consecutive frames relate. We use the **modified (Craig) Denavit–Hartenberg** convention, which needs only two axes per frame (x and z) and four numbers per joint:

$$(\alpha_{i-1}, \quad a_{i-1}, \quad d_i, \quad \theta_i)$$

symbol	name	meaning
α_{i-1}	link twist	angle from Z_{i-1} to Z_i about X_{i-1}
a_{i-1}	link length	distance between the two z -axes along X_{i-1}
d_i	link offset	distance from X_{i-1} to X_i along Z_i
θ_i	joint angle	angle from X_{i-1} to X_i about Z_i (the variable for a revolute joint)

Key point. For a revolute joint only θ_i changes as the motor turns; $\alpha_{i-1}, a_{i-1}, d_i$ are fixed by the robot's geometry.

Meaning. Twist and length describe how one joint axis sits relative to the next; offset and angle describe how far and how much you travel along/around the joint axis to reach the next link.

Chapter 2 — Forward kinematics

Question. Given the six joint angles, where is the tool?

Each pair of frames is related by one **modified-DH link transform** — exactly the matrix in your notes:

$${}^{i-1}_iT = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_{i-1} \\ s\theta_i c\alpha_{i-1} & c\theta_i c\alpha_{i-1} & -s\alpha_{i-1} & -s\alpha_{i-1}d_i \\ s\theta_i s\alpha_{i-1} & c\theta_i s\alpha_{i-1} & c\alpha_{i-1} & c\alpha_{i-1}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

with $c\theta = \cos\theta$, $s\theta = \sin\theta$. Chaining the six link transforms from the base to the tool gives the full forward kinematics:

$${}^0_N T = {}^0_1 T {}^1_2 T {}^2_3 T \cdots {}^{N-1}_N T$$

Key point. Forward kinematics is just a **product of these link matrices**. One set of joint angles gives exactly one tool pose.

Meaning. The top-left 3×3 block of the tool transform T is the tool's orientation; the top-right 3×1 block is its position.

In code (mycobot_thesis/robot.py): `_dh_link(alpha, a, d, theta)` builds one matrix above; `fk_frames(q)` multiplies them and returns every frame; and `forward_kinematics(q)` returns the last one, ${}^0_N T$. The DH table (identified from the URDF, verified to 10^{-15} m) is the array `DH_TABLE`.

Chapter 3 — The Jacobian (velocity-propagation method)

Question. How does the tool move when the joints move? The **Jacobian** $J(q)$ answers this, mapping joint speeds $\dot{q}$ to the tool's linear and angular velocity (its *twist*).

Your notes build J by **propagating velocity outward**, frame by frame. Angular velocity carries out and gains the new joint's spin:

$${}^{i+1}_i \omega_{i+1} = {}^{i+1}_i R^i \omega_i + \dot{\theta}_{i+1} {}^{i+1}_i \hat{Z}_{i+1}$$

Linear velocity carries out, picking up the lever-arm cross-product, and gains a prismatic slide (zero for our revolute joints):

$${}^{i+1}v_{i+1} = {}^{i+1}R({}^i v_i + {}^i \omega_i \times {}^i P_{i+1}) + \dot{d}_{i+1} {}^{i+1} \hat{Z}_{i+1}$$

Meaning. The rotation R carries a vector from frame i into frame $i+1$; the cross-product $\omega \times P$ is the velocity a rotating link imparts at its far end; $\dot{\theta}_{i+1}$ and $\dot{d}_{i+1}$ add the new joint's own motion.

For an all-revolute arm this closes into the compact **geometric Jacobian**, whose i -th column is

$$J_i = \begin{bmatrix} {}^0 \hat{Z}_i \times ({}^0 P_e - {}^0 P_i) \\ {}^0 \hat{Z}_i \end{bmatrix}$$

Key point. Column i says: *if only joint i spins, the tool's linear velocity is $\hat{Z}_i \times (P_e - P_i)$ and its angular velocity is $\hat{Z}_i$.*

In code (`robot.py`): `jacobian(q)` builds exactly these columns from the frame axes z_i and origins p_i returned by `fk_frames`. It is verified against a finite-difference Jacobian to $\approx 2.5 \times 10^{-7}$.

Chapter 4 — Inverse kinematics

Question. We want the tool at a desired pose — which joint angles get there? There is no simple formula, so we iterate with **damped least squares**:

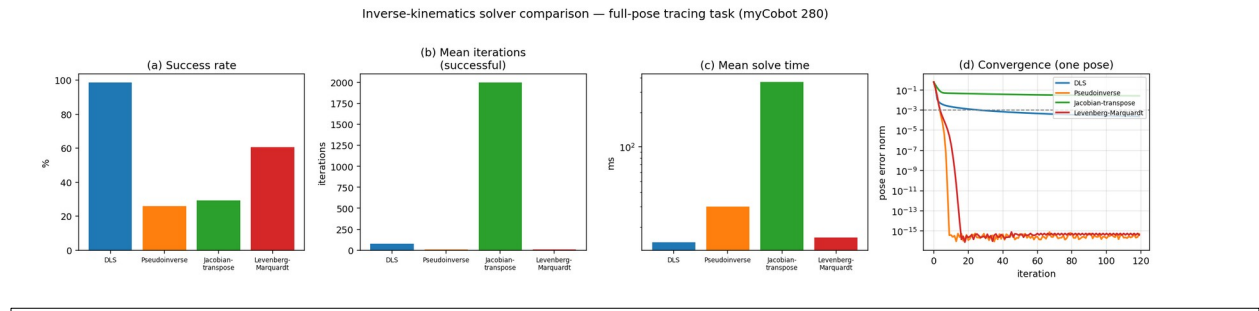
$$q_{k+1} = q_k + J^\top (JJ^\top + \lambda^2 I)^{-1} e$$

where e is the 6-vector pose error (position + orientation) and λ is a small damping constant.

Key point. The damping $\lambda^2 I$ is what keeps the step finite near a **singularity** (where J loses rank and a plain inverse blows up). It trades a tiny steady-state error for stability.

Meaning. Each step multiplies the pose error by a damped inverse of the Jacobian to get a joint correction, then repeats until the tool reaches the target.

In code (mycobot_thesis/ik.py): `damped_least_squares(...)` is the equation above; `pseudoinverse`, `jacobian_transpose`, and `levenberg_marquardt` are the alternatives compared in the thesis. Damped least squares was the most robust (98.7% success); see the figure below.



Chapter 5 — Manipulability and the task-constrained workspace

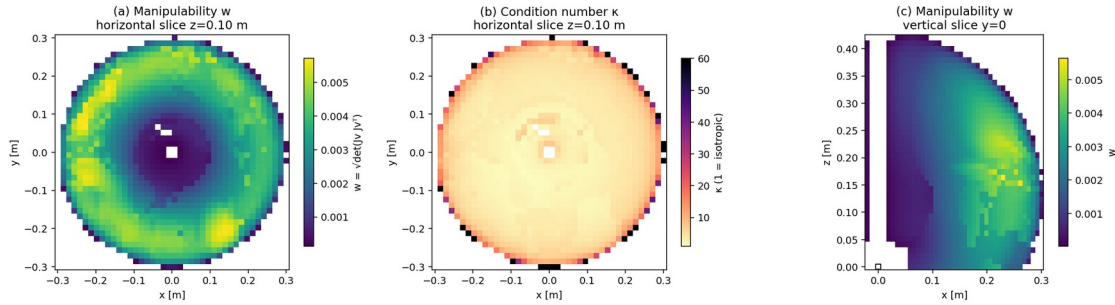
Not every reachable point is equally good, and — for drawing — not every reachable point can be reached with the pen **perpendicular to the surface**.

Dexterity. Yoshikawa manipulability and the condition number measure how far a configuration is from a singularity:

$$w = \sqrt{\det(J_v J_v^\top)}, \quad \kappa = \frac{\sigma_{\max}(J_v)}{\sigma_{\min}(J_v)}$$

Meaning. $w \rightarrow 0$ (and $\kappa \rightarrow \infty$) at a singularity: the arm loses a direction of motion and IK becomes twitchy. Large w , small κ = dexterous.

Manipulability and conditioning of the myCobot 280 (brighter = more dexterous, further from singularity)



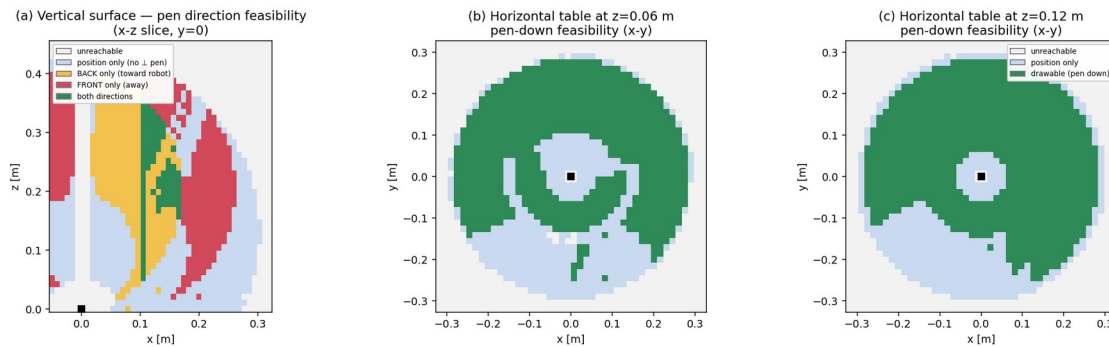
Drawable workspace. Given a surface with normal n , the positions the pen can reach *while staying perpendicular* form the task-constrained set

$$\mathcal{D}(n) = \{ p \in \mathcal{R} : \exists q, \text{fk}(q) = p, \hat{z}(q) = \pm n \}$$

Key point. The **reachable** set R and the **drawable** set $\mathcal{D}(n)$ are different: $\mathcal{D}(n) \subset R$, and the gap depends on the surface orientation.

For the myCobot: only 57.9% of R allows a perpendicular pen on a vertical surface (35.2% front-facing), and 63.5% allows pen-down drawing on a horizontal table — an **annulus** around a dead zone above the base.

Task-constrained workspace: where the myCobot 280 can hold the pen normal to the surface



Chapter 6 — Reading the code

Every module mirrors a chapter. The **library** is pure math (no ROS); the **nodes** wrap it for live RViz.

mycobot_thesis/robot.py — the model (Chapters 2–3, 5).

- **DH_TABLE** — the identified modified-DH parameters $(\alpha_{i-1}, a_{i-1}, d_i, \theta_i^{\text{off}})$.

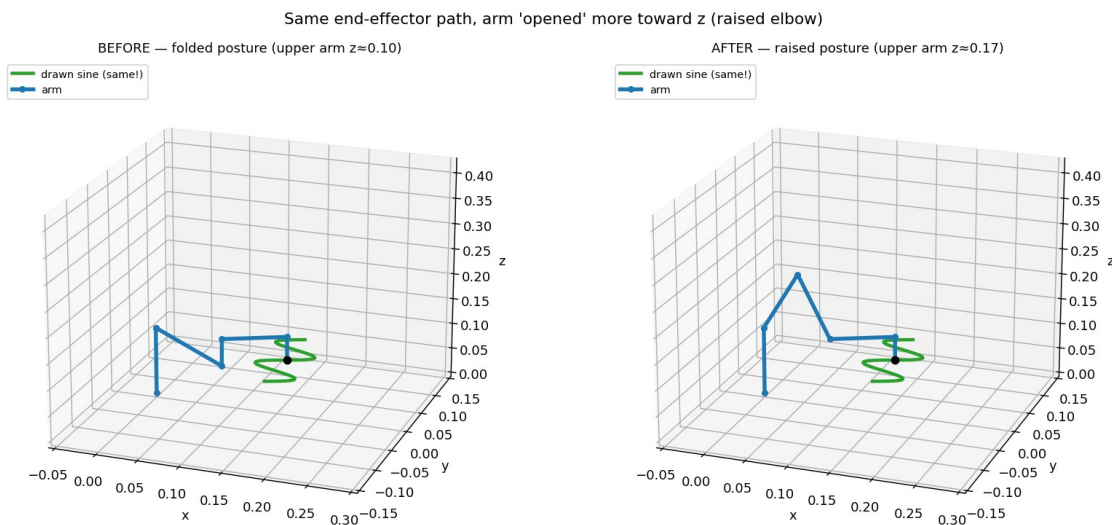
- `_dh_link`, `fk_frames`, `forward_kinematics` — the link matrix and the product.
- `jacobian` — the velocity-propagation columns.
- `manipulability`, `condition_number` — the dexterity metrics.
- `target_orientation(normal, advance)` — builds the pen-perpendicular target frame.

mycobot_thesis/ik.py — the solvers (Chapter 4). `damped_least_squares` and the three alternatives, each returning convergence info.

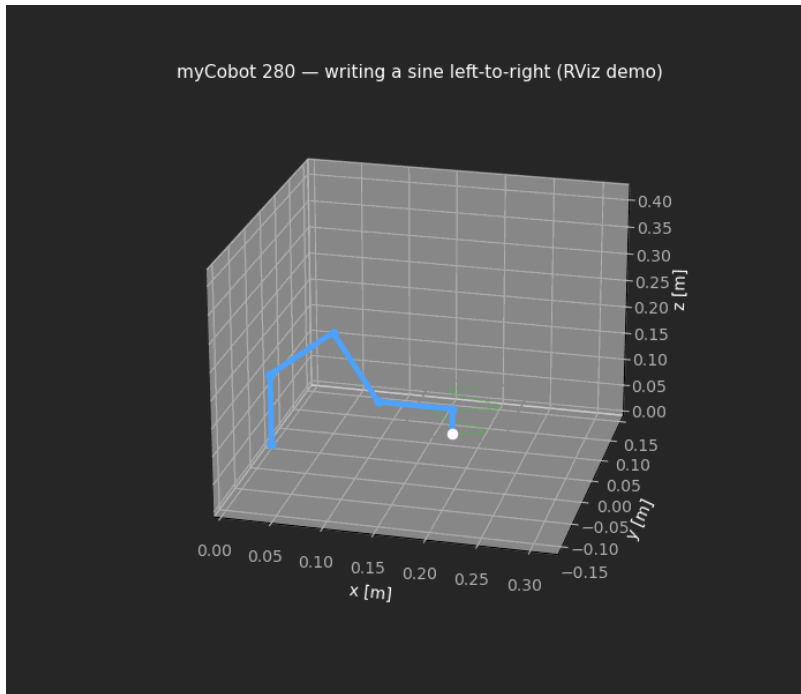
mycobot_thesis/draw_node.py — the ROS driver. It picks a surface preset, generates the sine targets, solves IK for each (warm-started), and streams `/joint_states`. The 'paper' preset uses a **raised** IK branch so the arm is opened toward *z* while the pen traces the identical sine.

Key point. A ROS node is a *thin wrapper*: it calls the same robot/ik library the offline analysis uses, then `publish()`s. Same math, live robot.

Redundancy in action. A 6-DOF arm can reach the same tool pose with different postures (IK branches). Seeding the IK toward a raised branch keeps the drawn sine identical while lifting the elbow:



The live demo (`ros2 launch mycobot_thesis thesis_draw.launch.py surface:=paper`) writes the sine left-to-right on a table in front of the robot, raised posture:



Appendix — Identified DH table and commands

i	α_{i-1} (°)	a_{i-1} (m)	d_i (m)	θ_i^{off} (°)
1	0	0	0.13056	+90
2	+90	0	0	-90
3	0	-0.1104	0	0
4	0	-0.096	0.06062	-90
5	+90	0	0.07318	+90
6	-90	0	0.0456	0

analysis (pure Python)

cd ~/ros2_ws/src/mycobot_thesis

python3 analysis/exp1_feasibility.py

workspace

python3 analysis/exp2_ik_comparison.py

live drawing in RViz

cd ~/ros2_ws && colcon build --packages-select mycobot_thesis &&
source install/setup.bash

bash src/mycobot_thesis/scripts/run_thesis_demo.sh surface:=paper

task-constrained

IK solver comparison